

JavaScript IIFE (Immediately Invoked Function Expression) - Cheat Sheet

What is an IIFE?

IIFE stands for Immediately Invoked Function Expression. It is a function that executes immediately after being defined.

Example:

```
(function () {  
    console.log("IIFE executed");  
})();
```

Why Use IIFE?

- Prevent global scope pollution
- Create private variables (encapsulation)
- Execute initialization code immediately
- Useful in closures and loop callbacks

Syntax

```
(function() {  
    // code  
})(); // Traditional
```

```
((() => {  
    // code  
}))(); // Arrow Function
```

```
(function namedIIFE() {  
    // code  
})(); // Named IIFE
```

Use Cases

1. Private State / Module Pattern:

```
const counter = (function () {  
    let count = 0;  
    return {  
        inc: () => ++count,  
        get: () => count  
    };  
})();
```

2. Loop Scope Fix:

```
for (var i = 0; i < 3; i++) {  
    (function(index) {  
        setTimeout(() => console.log(index), 1000);  
    })(i);  
}
```

3. Async IIFE:

JavaScript IIFE (Immediately Invoked Function Expression) - Cheat Sheet

```
(async () => {
  const data = await fetch('/api/data');
  console.log(await data.json());
})();
```

Function Declaration vs Expression

Type	Description
Declaration	function foo() {}
Expression	(function() {})()
IIFE Needs	Function Expression

Scope & Encapsulation

```
(function() {
  var secret = "hidden";
  console.log(secret); // OK
})();
console.log(secret); // Error: not defined
```

IIFE vs ES Modules

Feature	IIFE	ES6 Modules
Scope	Function Scope	Module Scope (file-based)
Encapsulation	Manual	Built-in
Imports/Exports	Not natively	Supported
Modern Usage	Rare	Recommended

Interview Highlights

- Basic:
- What is IIFE?
 - Syntax and Purpose
- Intermediate:
- Closure in loops with IIFE
 - Returning from IIFE
 - Named vs Anonymous IIFE
- Advanced:
- IIFE for Module Pattern
 - Async/Await IIFE
 - IIFE vs Modules
 - IIFE in Webpack or UMD